

Week 3: From Latches to Edge-Triggered Flip-Flops

Where We Stopped

A latch allows us to store one bit of information.

Input $\longrightarrow$ Latch $\longrightarrow$ Q

The **ENABLE** signal decides when the latch is allowed to respond to the input.

Question for This Week

Why is an **ENABLE-controlled latch** not sufficient for building synchronous digital systems?

Enable-controlled latch $\longrightarrow$ Clock $\longrightarrow$ Edge-triggered flip-flop

The Problem with an ENABLE-Controlled Latch

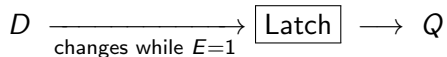
Consider a D latch:

$$E = 1 \Rightarrow Q \text{ follows } D$$

$$E = 0 \Rightarrow Q \text{ holds its previous value}$$

Problem

The latch remains **open for the entire duration** for which ENABLE is active. Therefore, if D changes while $E = 1$, the output Q may also change.



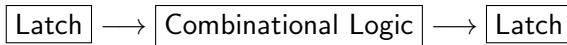
Long active interval

$\Rightarrow$

Input changes can propagate

Why Can This Become a Problem?

Suppose the output of one storage element is connected to combinational logic and then to another storage element.



If both latches are active for a sufficiently long interval,

new value $\longrightarrow$ logic $\longrightarrow$ next latch

may propagate during the **same active interval**.

Key Problem

We would prefer the state of the circuit to change at a **well-defined instant**, rather than throughout an ENABLE interval.

How Can We Prevent Continuous Changes?

We want the storage element to effectively accept a value **only once** during a clock cycle.

Two natural approaches are possible:

Approach 1: Isolate Input and Output

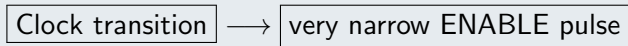
Use two storage stages.



While one stage is accepting data, the other stage holds the output.

Approach 2: Make the Active Interval Very Short

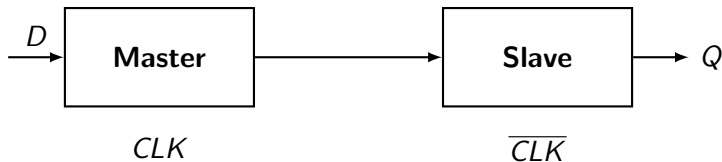
Instead of keeping ENABLE active for a long time,



so that the circuit has only a very short opportunity to change.

Approach 1: Master–Slave Structure

Use two latches controlled by complementary clock levels.



For example:

$CLK = 1 :$

Master active

Slave inactive

$CLK = 0 :$

Master inactive

Slave active

Main Idea

The output stage does not read the master while the master is changing. It reads the master's **stored value** after the master becomes inactive.

Limitation of the Master–Slave Approach

The external output changes only when information is transferred from Master to Slave. However, the **Master is still a level-sensitive latch**.

While the Master is active,

D changes $\Rightarrow Q_M$ may change

Therefore, a temporary value appearing at the input during the active interval may be captured by the Master.

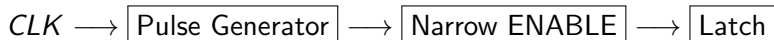
0-Catching / 1-Catching

An unwanted temporary 0 or 1 occurring during the Master's active interval may be captured and later transferred to the Slave.

Master–Slave $\neq$ true instantaneous sampling

Approach 2: Can We Make ENABLE Very Short?

Instead of allowing the latch to remain open for a large fraction of the clock cycle, generate a very narrow pulse.



Ideally,

$$T_{\text{enable}} \rightarrow \text{very small}$$

so the latch effectively observes the input only around a clock transition.

Desired Behaviour

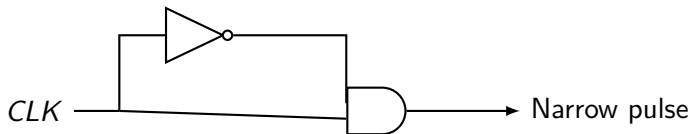
Long clock level



short sampling interval

Generating a Narrow Pulse

A simple idea is to use the propagation delay of an inverter.



Immediately after a clock transition, the direct and delayed versions of the clock temporarily have different values.

That short interval can be used to generate a pulse.

$$T_{\text{pulse}} \approx t_{pd, \text{gate}}$$

But Now We Have Another Problem

The narrow pulse exists because a gate does **not** respond instantaneously.

$$T_{\text{pulse}} \approx t_{pd,\text{gate}}$$

But propagation delay depends on physical implementation:

- transistor and gate characteristics,
- loading,
- supply voltage,
- temperature,
- manufacturing variation.

Problem

The sampling interval is now determined by **circuit-dependent propagation delay**.

Can we detect the clock edge **without intentionally using a gate delay to determine the pulse width?**

From Level Triggering to Edge Triggering

What we really want is not:

Respond while $CLK = 1$

or

Respond while $CLK = 0$

but rather:

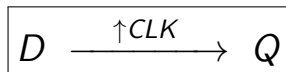
Respond when the clock changes state

For a positive-edge-triggered flip-flop:

$0 \rightarrow 1 \Rightarrow$ sample D

At other times,

Positive-Edge-Triggered D Flip-Flop



For a positive-edge-triggered D flip-flop,

$$Q(t^+) = D \quad \text{at } \uparrow CLK$$

Between clock edges,

$$Q(t^+) = Q(t)$$

Important Difference

The input is not continuously transferred during the HIGH clock level.
The value of D is captured around the **positive clock edge**.

How Can a Circuit “Remember” a Clock Edge?

Question:

If the clock remains HIGH after the rising edge,

$$CLK = 1$$

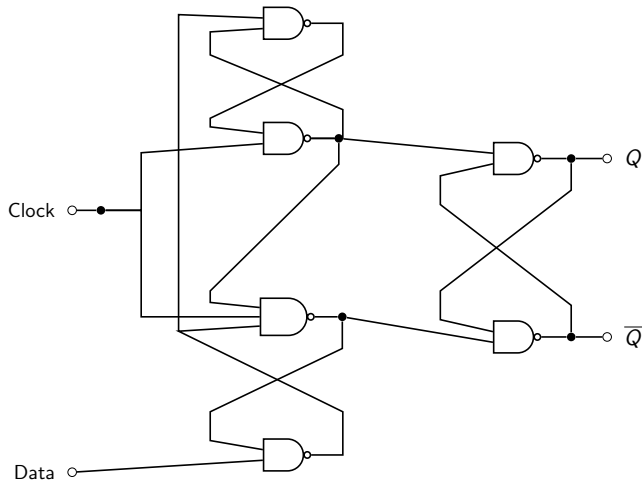
how does the flip-flop know that the edge has already occurred?

Key Idea

The circuit contains internal feedback.

The clock transition changes an internal state, and that state prevents subsequent changes in D from propagating to the output during the same clock level.

Positive-Edge-Triggered D Flip-Flop



Positive edge:

D is sampled at $\uparrow CLK$, $Q^+ = D$

Edge Triggering Solved One Problem – But Is That Enough?

We have designed a flip-flop that samples its input only around a clock edge.



What Edge Triggering Gives Us

The storage element no longer remains transparent throughout the entire active clock level.

New Question

Can D change at **any instant** close to the clock edge?

Can the clock frequency be made arbitrarily high?

No – physical circuits require time.

Timing Requirements of a Flip-Flop

For correct operation, several timing parameters must be satisfied.

Parameter	Meaning
t_w	Minimum required clock pulse width
t_{setup}	Input must be stable before the clock edge
t_{hold}	Input must remain stable after the clock edge
t_p	Time taken for the output to respond after the clock edge

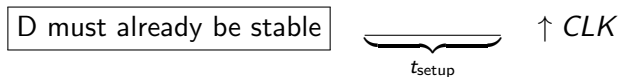
Important

An edge is an ideal mathematical instant.

A physical flip-flop needs a small **time window around that edge** to reliably capture the input.

Setup Time

Consider a positive-edge-triggered D flip-flop.



Setup Time

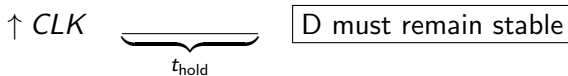
t_{setup} is the minimum time for which the input must remain stable **before** the active clock edge.

If the rising edge occurs at time t ,

D should be stable from approximately $t - t_{\text{setup}}$ to t .

Hold Time

Capturing the value does not finish exactly at the mathematical clock edge.



Hold Time

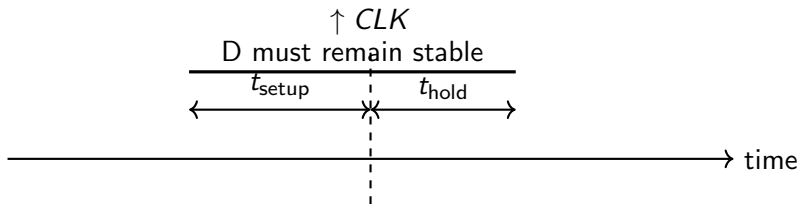
t_{hold} is the minimum time for which the input must remain stable **after** the active clock edge.

If the rising edge occurs at time t ,

D should remain stable until approximately $t + t_{\text{hold}}$.

Setup and Hold Define a Sampling Window

For a positive-edge-triggered flip-flop:



$$t - t_{setup} \leq \text{stable } D \leq t + t_{hold}$$

Important

Hold time does **not** mean that D must remain stable for the entire HIGH clock level. It only specifies a short interval **after the active edge**.

Propagation Delay of a Flip-Flop

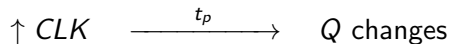
Suppose the flip-flop samples

$$D = 1$$

at the rising edge.

Does Q become 1 at exactly the same instant?

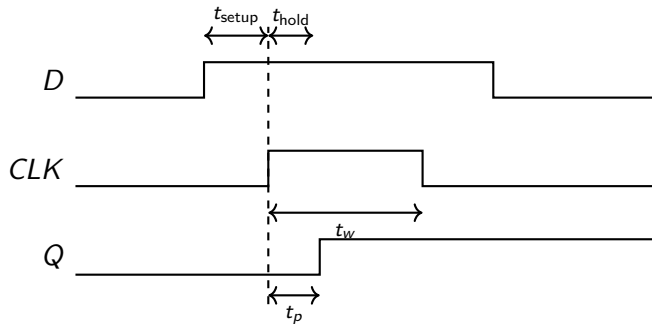
No.



Propagation Delay

Propagation delay is the time between the active clock edge and the corresponding change at the flip-flop output.

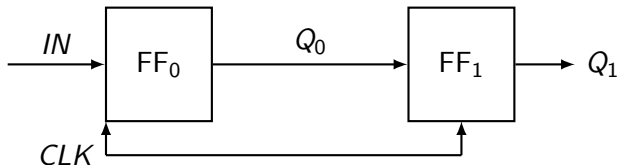
Timing of a Positive-Edge-Triggered D Flip-Flop



Input stable around edge + Output changes after t_p

What Happens When We Cascade Flip-Flops?

Consider two positive-edge-triggered D flip-flops driven by the same clock.



At the same rising clock edge:

- FF₀ samples *IN*.
- FF₁ samples the **current** value of *Q*₀.

Question

If FF₀ also changes at this edge, why does FF₁ not accidentally capture the new *Q*₀?

Propagation Delay Protects the Next Flip-Flop

At the active clock edge, FF_1 must keep seeing its old input for at least

$$t_{\text{hold}}.$$

But FF_0 does not change its output immediately.

$$\uparrow CLK \xrightarrow{t_p} Q_0 \text{ changes}$$

Therefore, if

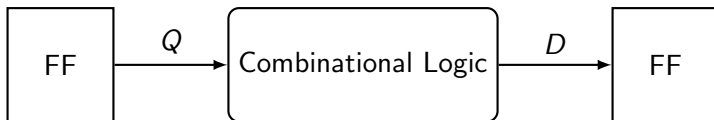
$$t_p > t_{\text{hold}}$$

then FF_1 finishes its required hold interval **before the new value from FF_0 arrives.**

$$\text{same clock edge} \Rightarrow FF_1 \text{ captures old } Q_0$$

The Real Processor Situation

Normally, flip-flops are not connected directly.
There is combinational logic between them.



After one clock edge:

$$FF_1 \xrightarrow{t_p} Q \xrightarrow{t_{\text{logic}}} D_{FF_2}$$

The result must arrive early enough to satisfy the setup time of FF_2 before the **next** clock edge.

What Determines the Clock Period?

At clock edge t : FF_1 launches new data

The data requires: t_p

to leave FF_1 , and then t_{logic}

to pass through the combinational circuit.

Before the next edge at $t + T$, FF_2 requires

t_{setup}

of stable input.

Therefore,

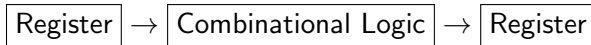
$$T \geq t_p + t_{\text{logic}} + t_{\text{setup}}$$

This Is Why Clock Frequency Cannot Be Arbitrarily High

$$f_{\text{max}} \approx \frac{1}{T_{\text{min}}}$$

What Really Determines Processor Clock Speed?

The clock frequency is therefore not determined by the flip-flop alone.



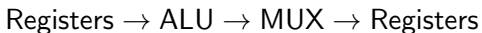
The minimum clock period depends on the slowest register-to-register path:

$$T_{\min} \geq t_p + t_{\text{logic,max}} + t_{\text{setup}}$$

Critical Path

The path having the largest combinational delay between two storage elements is called the **critical path**.

For example, the critical path may pass through:



Frequency Division using a Flip-Flop

Suppose the input clock has frequency f . Can we generate a new clock of frequency $\frac{f}{2}$?

Using a Toggle Flip-Flop

For a T flip-flop, set

$$T = 1.$$

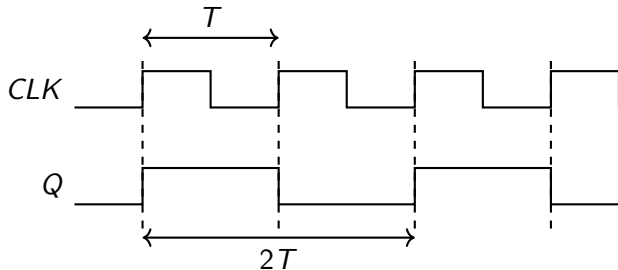
The output toggles at every active clock edge:

$$Q^+ = \overline{Q}.$$

Therefore, two input clock edges are required to complete one output cycle:

$$f_Q = \frac{f_{CLK}}{2}$$

Why Does the Frequency Become $f/2$?



The output changes only once at each active edge:

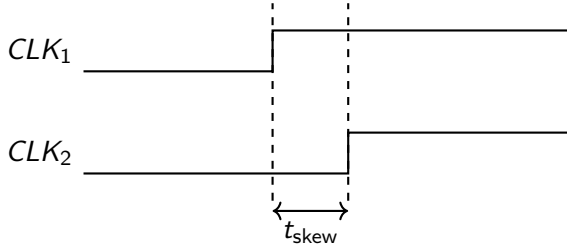
$$0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow \dots$$

Hence,

$$T_Q = 2T_{CLK}$$

$$f_Q = \frac{f_{CLK}}{2}$$

Clock Skew: Timing View



The source clock generated only **one rising edge**. However,

FF_1 sees it first

FF_2 sees it t_{skew} later.

Important

Clock skew is not a different clock frequency.

It is a difference in the **arrival time of the same clock edge**.

Example: Clock Skew

Suppose the clock reaches two flip-flops through different paths.

$$t_{CLK,1} = 2 \text{ ns}$$

$$t_{CLK,2} = 3 \text{ ns}$$

Therefore,

$$t_{skew} = t_{CLK,2} - t_{CLK,1}$$

$$t_{skew} = 3 - 2 = 1 \text{ ns}$$

Thus, although both flip-flops are connected to the same clock,

FF_2 receives the active edge 1 ns later

Clock Skew and Minimum Clock Period

Without considering clock skew, we used

$$T_{\min} = t_{pFF} + t_{pCOMB} + t_{\text{setup}}.$$

But the clock edges may not arrive simultaneously.

To safely tolerate a maximum clock skew t_{skew} , we include it in the timing budget:

$$T_{\min} = t_{pFF} + t_{pCOMB} + t_{\text{setup}} - t_{\text{skew}}$$

Therefore,

$$f_{\max} = \frac{1}{T_{\min}}.$$

Effect

Larger clock-skew tolerance can increase the required clock period and therefore reduce the maximum safe clock frequency.

Example: Effect of Clock Skew on Clock Frequency

Suppose

$$t_{pFF} = 5 \text{ ns}, \quad t_{pCOMB} = 2 \text{ ns}, \quad t_{\text{setup}} = 3 \text{ ns}.$$

Without clock skew,

$$T_{\min} = 5 + 2 + 3 = 10 \text{ ns}, \quad f_{\max} = 100 \text{ MHz}.$$

Now suppose

$$t_{\text{skew}} = 2 \text{ ns}.$$

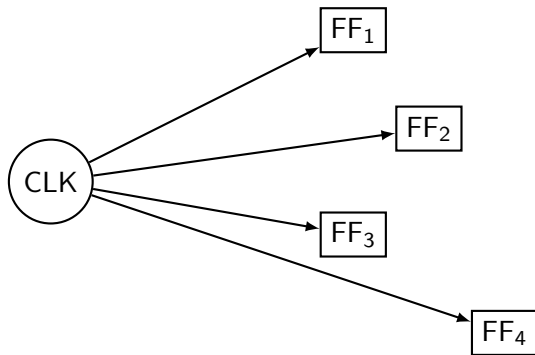
Then

$$T_{\min} = 5 + 2 + 3 - 2 = 8 \text{ ns},$$

$$f_{\max} = \frac{1}{8 \text{ ns}} = 125 \text{ MHz}$$

Why Does Clock Skew Occur Physically?

A processor may contain thousands or millions of storage elements.
The clock must be distributed across the chip.



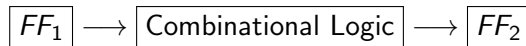
These paths may differ in:

- wire length, number of buffers, capacitive load, propagation delay.

Hence, the clock edges do not necessarily arrive everywhere at exactly the same instant.

Positive and Negative Clock Skew

Consider data travelling from FF_1 to FF_2 .



$$t_{\text{skew}} = t_{CLK,2} - t_{CLK,1}.$$

Positive Skew

$$t_{CLK,2} > t_{CLK,1}$$

The clock reaches the receiving flip-flop FF_2 later.

Negative Skew

$$t_{CLK,2} < t_{CLK,1}$$

The clock reaches the receiving flip-flop FF_2 earlier.

Hazards in Digital Circuits

A **hazard** is a momentary unwanted change in the output caused by unequal propagation delays along different logic paths.

Static Hazard: The initial and final output values are the same, but the output changes momentarily.

$$0 \rightarrow 1 \rightarrow 0 \quad \text{or} \quad 1 \rightarrow 0 \rightarrow 1$$

Example:

$$F = AS + \overline{S}B$$

Let $A = B = 1$. Then $F = S + \overline{S} = 1$.

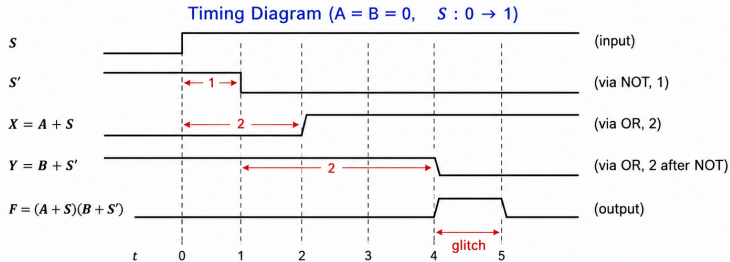
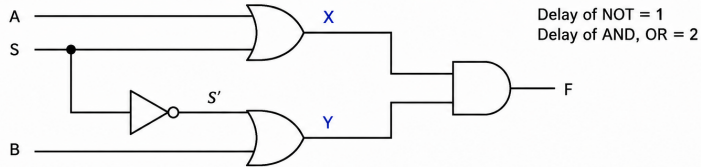
Ideally, when S changes, F should remain 1.

However, S and $\overline{S}$ travel through paths with different propagation delays, so the output may temporarily become

$$F : 1 \rightarrow 0 \rightarrow 1$$

This temporary glitch is a **static-1 hazard**.

Static Hazard Example



$F : 0 \rightarrow 1 \rightarrow 0$ (Static-0 Hazard)

From Flip-Flops to Registers

A flip-flop stores **one bit** of information.

To store an n -bit binary word, we use n flip-flops with a **common clock**.

Register = Group of Flip-Flops with a Common Clock
--

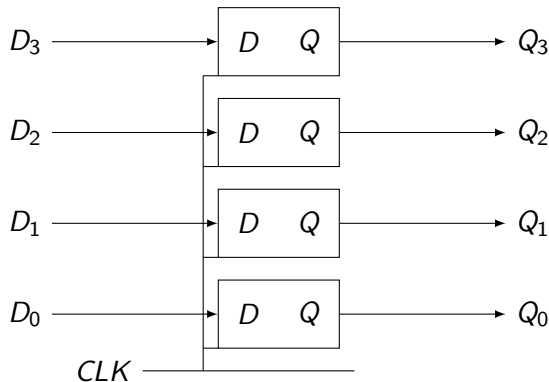
For example, a 4-bit register stores $D_3D_2D_1D_0$ using four flip-flops.

Depending on how data enters and leaves the register:

Type	Data Transfer
PIPO	Parallel-In Parallel-Out
SISO	Serial-In Serial-Out
PISO	Parallel-In Serial-Out
SIPO	Serial-In Parallel-Out

Basic Parallel-In Parallel-Out (PIPO) Register

A 4-bit PIPO register can be constructed using **four D flip-flops** with a common clock.



At the active clock edge,

$$Q_3 Q_2 Q_1 Q_0 \leftarrow D_3 D_2 D_1 D_0$$

Why Do We Need a LOAD Signal?

In a synchronous system, the clock keeps running continuously.

For the basic PIPO register,

Every active clock edge $\Rightarrow$ new input is stored

But what if we want the register to **keep its current value**?

We introduce a control signal: LOAD

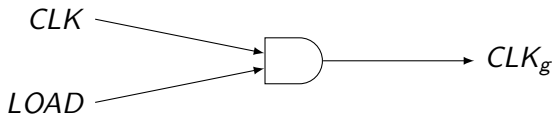
LOAD	Required Operation
0	Hold the current value
1	Load the new input

$$Q^+ = \begin{cases} Q, & \text{LOAD} = 0 \\ D, & \text{LOAD} = 1 \end{cases}$$

Can We Simply Gate the Clock?

One possible idea is to combine **CLK** and **LOAD**:

$$CLK_g = CLK \cdot LOAD$$



If *LOAD* = 0, no clock edge reaches the flip-flops.

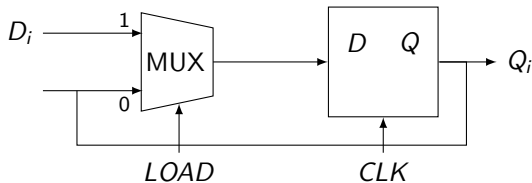
If *LOAD* = 1, the clock is allowed to reach the flip-flops.

Problem

Directly gating the clock with another signal can cause **timing problems**.

Implementing LOAD Using a Multiplexer

Instead of gating the clock, place a **2-to-1 MUX** before the D flip-flop.



$$Q_i^+ = \begin{cases} Q_i, & LOAD = 0 \\ D_i, & LOAD = 1 \end{cases}$$

How Does the LOAD Circuit Work?

The clock continues to reach the flip-flop **regardless of LOAD**.

Case 1: $LOAD = 1$

The MUX selects the external input: $D_{FF} = D_i$, At the next active clock edge,

$$Q_i^+ = D_i$$

The new value is stored.

Case 2: $LOAD = 0$

The MUX selects the current output through feedback: $D_{FF} = Q_i$, At the next active clock edge,

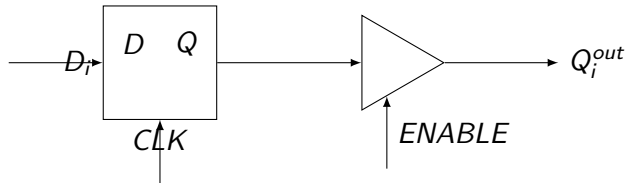
$$Q_i^+ = Q_i$$

The register keeps its previous value.

Adding an ENABLE Input

The register may store a value, but we may not always want that value to drive the output bus.

For this purpose, an **ENABLE** signal can be added at the output.



$$Q_i^{out} = \begin{cases} Q_i, & ENABLE = 1 \\ Z, & ENABLE = 0 \end{cases}$$

where Z represents the **high-impedance state**.

What is the High-Impedance State?

A tri-state output can have three possible states:

0,	1,	Z
----	----	---

Z is called the **high-impedance state**.

When the output is in state Z, the device effectively disconnects from the bus

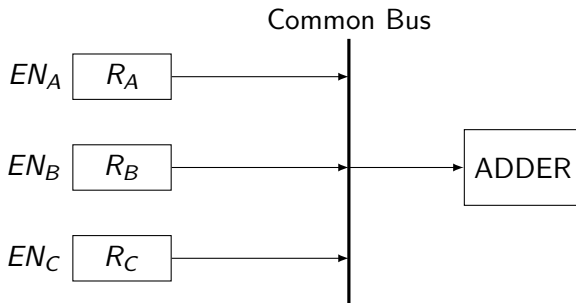
For a tri-state buffer:

ENABLE	Input	Output
0	X	Z
1	0	0
1	1	1

Therefore, several devices can be connected to the same bus, provided that only the required device is enabled.

Why Do Registers Need Output ENABLE?

Consider several registers connected to a **common bus**.

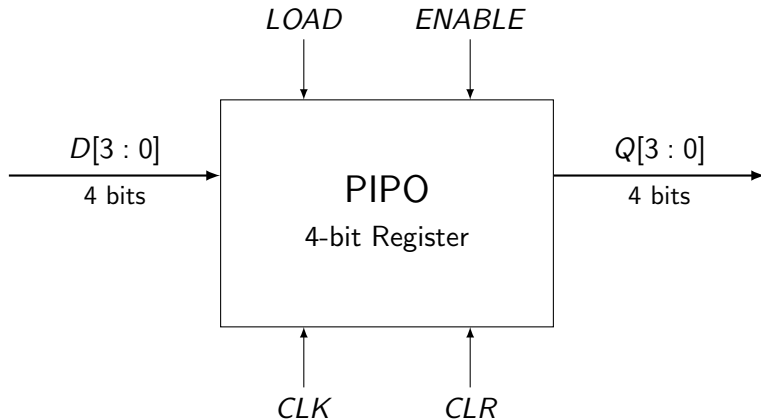


Only **one register** should drive the bus at a time.

$$EN_A = 1 \quad \Rightarrow \quad R_A \text{ drives the bus}$$

while the other register outputs remain in state Z . If multiple registers drive different values onto the same bus simultaneously, a bus conflict can occur.

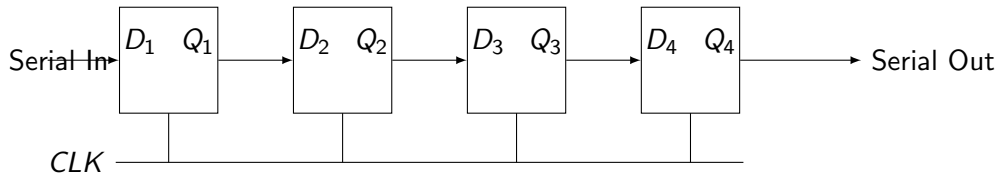
PIPO Register with Control Inputs



CLR	Clears the register
$LOAD$	Load new data / retain old data
$ENABLE$	Drive output / high impedance

Shift Register

A **shift register** is a register in which the stored binary data can be shifted one position at every active clock edge.



At every active clock edge,

$$Q_1^+ = SI, \quad Q_2^+ = Q_1, \quad Q_3^+ = Q_2, \quad Q_4^+ = Q_3.$$

Thus,

new bit enters $\rightarrow$ data shifts $\rightarrow$ one bit leaves

Example of Shifting

Consider the initial contents: $Q_1 Q_2 Q_3 Q_4 = 0101$.

Suppose the serial input is $SI = 1$.

At the next active clock edge,

SI	Q_1	Q_2	Q_3	Q_4
1	0	1	0	1

becomes

Q_1^+	Q_2^+	Q_3^+	Q_4^+
1	0	1	0

Therefore,

$0101 \longrightarrow 1010$

The old value of Q_4 is shifted out.

SHIFT Enable

Do we always want the contents of the register to shift at every clock edge?

Introduce a control signal:

<i>SHIFT</i>	
SHIFT	Operation at Active Clock Edge
0	Hold current contents
1	Shift by one position

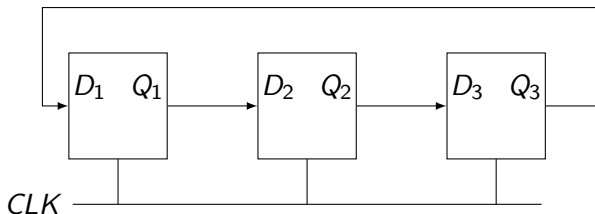
For each stage,

$$Q_i^+ = \begin{cases} Q_i, & SHIFT = 0 \\ Q_{i-1}, & SHIFT = 1 \end{cases}$$

with the first stage receiving the serial input.

Ring Counter

A **ring counter** is obtained from a shift register by feeding the last output back to the first input.



The last stored bit is therefore **recirculated**:

$$D_1 = Q_3$$

Typically the register is initialized with one 1 and the remaining bits 0.

3-bit Ring Counter

Suppose

$$Q_1 Q_2 Q_3 = 100.$$

At every clock edge, the single 1 circulates through the register:

$$100 \rightarrow 010 \rightarrow 001 \rightarrow 100 \rightarrow \dots$$

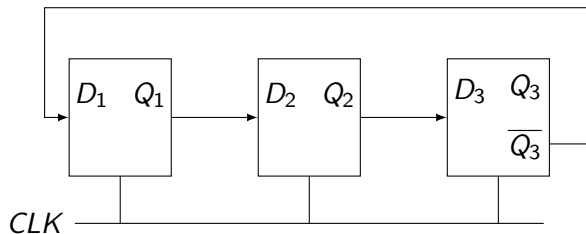
Clock	$Q_1 Q_2 Q_3$
0	100
1	010
2	001
3	100

For a k -bit ring counter, number of states= k

Therefore, Ring Counter is a Modulo- k counter

Twisted Ring or Johnson Counter

A **Johnson counter** is similar to a ring counter, but the **complement** of the last output is fed back.



$$D_1 = \overline{Q_3}$$

Unlike a ring counter, a Johnson counter may be initialized to

$$000 \dots 0.$$

4-bit Johnson Counter

Start with

$$Q_1 Q_2 Q_3 Q_4 = 0000.$$

The complemented last bit is shifted back into the register:

0000 $\rightarrow$ 1000

$\rightarrow$ 1100

$\rightarrow$ 1110

$\rightarrow$ 1111

$\rightarrow$ 0111

$\rightarrow$ 0011

$\rightarrow$ 0001

$\rightarrow$ 0000

For a k -bit Johnson counter,

Bidirectional Shift Register

A normal shift register shifts data in only one direction.

A **bidirectional shift register** can shift

left or right

depending on a control input.

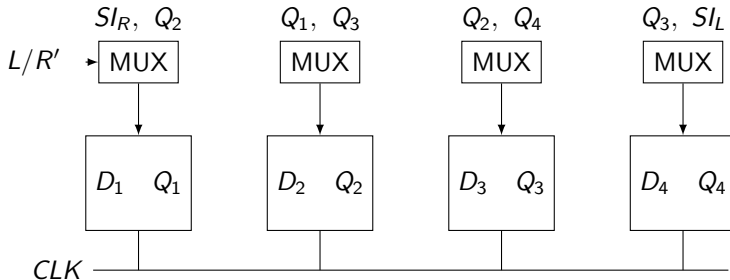
L/R'	Operation
0	Shift Right
1	Shift Left

How can the direction be selected?

Place a MUX before every D flip-flop

The MUX selects which neighbouring bit is connected to the D input.

4-bit Bidirectional Shift Register



Each MUX chooses between the two possible directions.

$L/R' \rightarrow \text{select neighbour} \rightarrow D_i$

Universal Shift Register

A **universal shift register** combines several register operations in a single circuit.

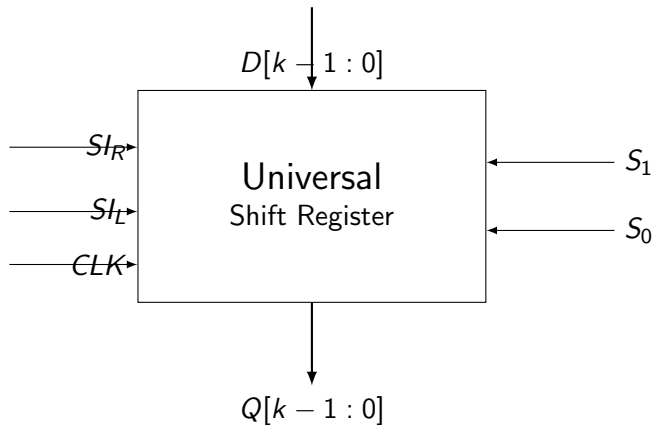
Two control inputs,

$$S_1, S_0,$$

select the required operation.

S_1	S_0	Operation
0	0	No change
0	1	Shift right
1	0	Shift left
1	1	Parallel load

Universal Shift Register: Block Diagram



Serial Input + Parallel Input + Bidirectional Shift

How is a Universal Shift Register Built?

Each flip-flop is preceded by a **4-to-1 multiplexer**.

The MUX selects one of four possible inputs:

S_1	S_0	MUX Selects
0	0	Current Q_i
0	1	Bit from left neighbour
1	0	Bit from right neighbour
1	1	Parallel input D_i

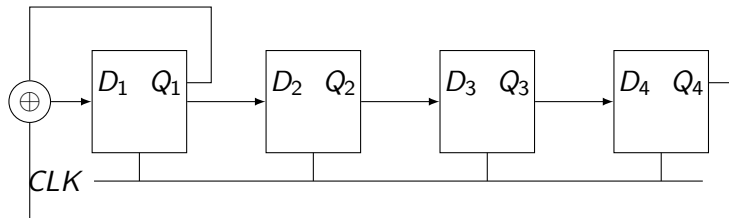
Therefore, for each bit,

4-to-1 MUX $\rightarrow$ D Flip-Flop
--

The same control signals S_1S_0 are connected to the multiplexers of **all stages**.

Linear Feedback Shift Register (LFSR)

A **Linear Feedback Shift Register (LFSR)** is a shift register in which the serial input is generated from some of the register outputs.



For the feedback shown,

$$D_1 = Q_1 \oplus Q_4$$

and

$$Q_2^+ = Q_1, \quad Q_3^+ = Q_2, \quad Q_4^+ = Q_3.$$

LFSR: Example State Sequence

Consider the 4-bit $LFSRD_1 = Q_1 \oplus Q_4$
with initial state $Q_1 Q_2 Q_3 Q_4 = 0001$

At each active clock edge, $Q_1^+ = Q_1 \oplus Q_4$, and all other bits shift one position.

Clock	$Q_1 Q_2 Q_3 Q_4$
0	0001
1	1000
2	1100
3	1110
4	1111
5	0111
6	1011
7	0101

The generated pattern depends on the **feedback taps** selected for the XOR.

LFSR: Properties and Application

- With proper feedback taps, an n -bit LFSR can generate

$$2^n - 1$$

distinct states before repeating.

- The generated sequence appears random but is deterministic.
- Hence, LFSRs are used for **pseudo-random number generation**.
- The all-zero state is not used:

$$00 \dots 0 \rightarrow 00 \dots 0$$

since XOR feedback remains 0.

Shift Register Variations

Type	Main Idea
Shift Register	Data shifts between successive stages
Ring Counter	Q_{last} fed back to first stage
Johnson Counter	$\overline{Q_{\text{last}}}$ fed back to first stage
Bidirectional Register	Can shift either left or right
Universal Register	Hold, shift left/right, parallel load
LFSR	XOR of selected outputs used as feedback

Shift Registers + Feedback / MUX $\Rightarrow$ Many Sequential Structures

Binary Counter

A **binary counter** is a set of flip-flops whose state changes in response to input clock pulses.

The combined state of the flip-flops represents the **number of pulses** received.

For a 4-bit counter:

$$0000 \rightarrow 0001 \rightarrow 0010 \rightarrow 0011 \rightarrow \dots$$

Counters can be classified as:

Type	Clocking
Asynchronous	Flip-flops do not share the same clock
Synchronous	All flip-flops share the same clock

Asynchronous Counter (Ripple Counter)

In an **asynchronous counter**, only the first flip-flop receives the external clock.

The output of one flip-flop provides the clock for the next flip-flop.

$$CLK \rightarrow FF_0 \rightarrow FF_1 \rightarrow FF_2 \rightarrow FF_3$$

Therefore, the flip-flops do **not** change state simultaneously.

The change propagates from one stage to another:

Ripple Effect

Ripple counters are simple and require relatively little hardware.

4-bit Ripple Up Counter

Consider binary up-counting:

0		0000	4		0100
1		0001	5		0101
2		0010	6		0110
3		0011	7		0111

Observe what happens during a carry:

0001 $\rightarrow$ 0010

0011 $\rightarrow$ 0100

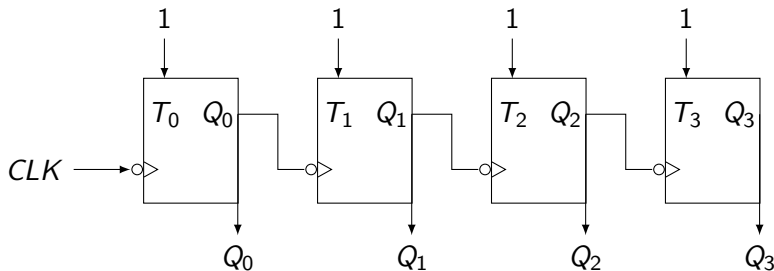
0111 $\rightarrow$ 1000

Whenever a bit changes

1 $\rightarrow$ 0

the **next higher bit toggles**.

4-bit Ripple Up Counter



For negative-edge-triggered T flip-flops,

$$Q_i \rightarrow CLK_{i+1}$$

so the next stage toggles when

$$Q_i : 1 \rightarrow 0.$$

Frequency Division in a Ripple Counter

Suppose the input clock frequency is

$$f_{CLK} = f.$$

Each T flip-flop toggles at half the frequency of the previous stage.

Signal	Frequency
Q_0	$\frac{f}{2}$
Q_1	$\frac{f}{4}$
Q_2	$\frac{f}{8}$
Q_3	$\frac{f}{16}$

Therefore, an n -bit counter has 2^n states and is also called a Modulo- 2^n counter or a Divide-by- 2^n counter

Why Is It Called a Ripple Counter?

The external clock is applied only to the first flip-flop.

$$CLK \rightarrow FF_0 \rightarrow FF_1 \rightarrow FF_2 \rightarrow FF_3$$

Each flip-flop changes only after the previous flip-flop changes.

Therefore, the state transition propagates through the counter stage by stage.

The transition “ripples” through the counter

Hence, the flip-flops do **not** change simultaneously.

4-bit Ripple Down Counter

Consider binary down-counting:

$1111 \rightarrow 1110 \rightarrow 1101 \rightarrow 1100 \rightarrow \dots$

Observe the transitions

$1110 \rightarrow 1101$

$1100 \rightarrow 1011$

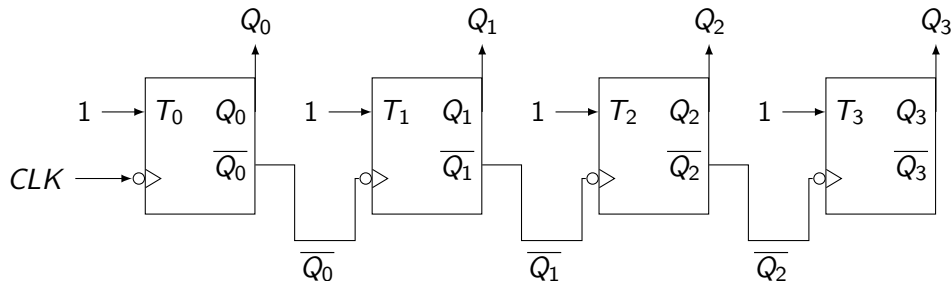
$1000 \rightarrow 0111.$

Whenever a bit changes

$0 \rightarrow 1$

the **next higher bit must toggle.**

4-bit Ripple Down Counter



For negative-edge-triggered T flip-flops,

$$\boxed{\overline{Q_i} \rightarrow CLK_{i+1}}$$

because the next stage toggles for $Q_i : 0 \rightarrow 1$.

Ripple Up Counter vs Down Counter

For **negative-edge-triggered** T flip-flops:

Counter	Next Clock From	Required Transition
Up	Q_i	$1 \rightarrow 0$
Down	$\overline{Q_i}$	$0 \rightarrow 1$

Thus,

$$\begin{array}{l} \text{Up Counter: } Q_i \rightarrow CLK_{i+1} \\ \text{Down Counter: } \overline{Q_i} \rightarrow CLK_{i+1} \end{array}$$

In both circuits,

$$T = 1$$

so every flip-flop operates in toggle mode.

- M. Morris Mano and Michael D. Ciletti, *Digital Design*.
 - Chapter 5: Synchronous Sequential Logic
 - Chapter 6: Registers and Counters
- ChatGPT, OpenAI, for assistance in preparing and refining lecture material.